AI 辅助研发赋能

从大模型演进、AI Agent 到 OpenSpec 需求开发实践

Author: My ChatGPT and Me

提纲 / Outline

- 01 AI 发展与代码辅助开发演进
- 02 AI Agent：概念、机制与实践
- 03 OpenSpec：规格驱动需求开发
- 04 赋能总结：价值、误区与拥抱方式
- 05 AI Agent 协作开发范式

ai_assisted_rd_enablement.md

Prompt　Agent　Review

Context　OpenSpec

Prompt → Context → Tools → Human Review

- 让 AI 多干活，但别让 AI 独自拍板
- 先澄清需求，再让 Agent 执行
- 用规范、测试与 Review 驯服 AI

本次目标：带大家入门 AI 辅助研发，把工具真正用起来

SECTION

第一部分 | AI 发展与代码辅助开发演进

从 ChatGPT 问世到 Coding Agent 逐步走进研发工作流

从知识到流程，从工具到方法

过去三年：AI 从“会对话”走向“会执行”

大模型能力完成四次连续跃迁，逐步从自然语言入口进入真实工作流

核心变化不是“AI 更会写答案”，而是它开始理解更复杂的上下文、调用工具、参与工作流，并围绕目标持续推进任务。

01

对话式入口化

2022.11 • ChatGPT 发布

AI 终于“说人话”了

上线约 2 个月达到约 1 亿月活用户；大模型第一次以聊天窗口的形式站到每个人面前。

02

多模态与长上下文化

2023 • GPT-4 / GPT-4V

AI 开始“看得更多、记得更长”

图片、代码、长文档、日志和报错信息可以被放进同一个分析空间，输入从一句话变成一个工作场景。

03

工具调用与工作流化

2023 • Plugins / Function Calling

AI 不只回答，还能“动手”

从告诉你怎么做，变成查资料、处理文件、运行代码、连接外部系统，进入具体工作流。

04

Agent 化执行

2024–2026 • Coding Agent

AI 开始围绕目标规划并执行

从“帮我写函数”升级为“读仓库、定位问题、改代码、跑测试、提交可 review 的变更”。

一句话收束：AI 的价值正在从“生成内容”，转向“完成任务”。

02

代码辅助开发：从“自动补全”到“任务托管”

代码编程是生成式 AI 最早深入影响的领域之一：从帮你写几行代码，走向理解项目、协助决策、执行工程任务。

本页讲什么

AI 编程工具的演进，本质上是开发者与工具关系的变化。

从“更聪明的 IDE 提示”，
到“随叫随到的代码搭子”，
再到“读过项目文档的新同事”，
最后走向“可托管小任务的实习工程师”。

一句话

AI 的工作单位，正在从“一段代码”升级为“一个任务”。

四个阶段：用程序员日常场景理解

每一步都不是“更会聊天”，而是更贴近真实开发动作

01	**补全时代** 行级 / 函数级补全、样板代码、API 调用建议	**更聪明的 IDE 提示**	写下 def get_user_by_id(...)，AI 已经猜到要查数据库、判空、返回用户对象。 **从“提示变量名”到“补完一段逻辑”。**
02	**解释 + 多工具时代** 解释代码、分析报错、生成测试、辅助重构	**随叫随到的代码搭子**	接手七层 if-else 的祖传代码，直接问：这段在做什么？哪里有风险？能否拆函数并补单测？ **从“帮我写代码”到“帮我读、改、测”。**
03	**代码库级理解** 结合仓库结构、依赖关系、Issue、错误栈和调用链	**读过项目文档的新同事**	线上 NullPointerException，不只看当前文件，而是顺着调用链找入口、相关文件和潜在影响面。 **从“看一段代码”到“看一个项目”。**
04	**Agent 化开发** 读仓库、拆任务、改代码、跑测试、修失败、整理 PR 说明	**可托管小任务的实习工程师**	一句“给接口加分页参数，兼容原调用并补单测”，AI 尝试读代码、改实现、跑测试、产出变更。 **从“一段代码”到“一个可 Review 的任务”。**

03

四个阶段背后的本质变化

AI 编程工具的演进，不只是“写代码更快”，而是开发者与工具的关系正在变化：从提示、协作，到任务托管。

核心变化

AI 的工作单位，正在从 “token / 函数” 变成 “变更集 / 任务 / 问题单”。

越往后，人类越需要把精力放在目标定义、架构判断、质量审查与风险控制上。

01

补全时代

IDE 提示升级版

开发者关系

AI 提示，我来判断

单位： 行 / 函数

接受补全、改样板代码、少查文档

从“记语法”转向“判断建议是否靠谱”

02

解释 + 多工具

随叫随到的代码搭子

开发者关系

AI 分析，我来决策

单位： 代码片段 / 文件

读祖传代码、解释报错、生成测试、重构函数

从“单兵硬读”转向“带着 AI 一起拆问题”

03

代码库级理解

读过项目文档的新同事

开发者关系

AI 理解项目，我来审查

单位： 模块 / 调用链

关联仓库结构、Issue、日志、依赖和影响面

从“局部修补”转向“工程上下文下的判断”

04

Agent 化开发

可托管小任务的实习工程师

开发者关系

AI 执行任务，我来验收

单位： 变更集 / 问题单

拆任务、改代码、跑测试、修失败、整理 PR

从“亲自写每一步”转向“定义目标并验收结果”

讲给团队听的落点： AI 越像“会做事的同事”，开发者越要像“任务负责人”：定义目标、拆清边界、验收质量。

SECTION

第二部分 | AI Agent：概念、机制与实践

从模型与上下文，到 opencode CLI、MCP 与 Agent 工作流

从知识到流程，从工具到方法

从“回答问题”到“执行任务”

Agent 不是“更会聊天的机器人”，
而是一个能围绕目标拆解步骤、调用工具、执行动作，并根据结果继续调整的系统。

一句话定义

目标导向 + 上下文理解 + 工具调用 + 反馈循环

从“给答案”升级为“尝试把任务往前推进”。

Agent 通常由这些能力组成

- 目标：知道要完成什么
- 上下文：读代码/文档/日志
- 计划：拆成可执行步骤
- 工具：搜索/文件/终端/API
- 执行：改代码/跑命令
- 反馈：根据结果修正

关键边界

Agent 能执行，不代表能负责；
需求澄清、架构取舍、质量风险和上线决策，仍然需要开发者把关。

同一个需求，普通助手和 Agent 的差别

例子：给用户列表接口增加分页能力

普通 AI 助手

你问一句，它答一句

“帮我写一个分页查询函数。”

返回一段代码建议

复制 / 改造 / 运行 / 测试
仍主要由开发者完成

AI Agent

你给目标，它推进一串工程动作

“兼容旧调用，补单测，跑测试，并整理变更说明。”

1	2	3	4	5	6
读仓库	**拟计划**	**改代码**	**补测试**	**跑测试**	**交付**
定位接口/调用方	列出修改范围	实现分页逻辑	覆盖边界用例	失败后继续修	生成 PR 说明

类比

普通助手像“答疑同学”；Agent 更像“可接小任务的实习生”。

以 opencode CLI 为例：/init 让 Agent 读懂项目

第 5 页重点：npm 安装、/init 初始化、AGENT.md 的作用，以及“先分析、先出计划”的日常使用方式。

01 安装与启动

```
npm install -g opencode-ai
opencode --version
```

```
cd /path/to/java-service
opencode
```

02 简单使用例子

先不要直接改代码，先让 Agent 看懂现状：

“帮我分析 UserController 中用户列表接口的调用链，说明会涉及哪些 Service / DTO / Repository，先不要修改代码。”

03 再进入实现

更稳的节奏：

分析现状 → 输出计划 → 确认风险 → 再让 Agent 修改代码与补测试。

不要一上来就说“直接帮我改完”。

为什么要先执行 /init?

/init 扫描项目 → **生成/更新** AGENT.md → **沉淀** 项目上下文

AGENT.md 可以理解为：写给 AI Agent 的项目说明书。

它让 Agent 每次进入任务时，不是“通用助手”，而是带着项目角色、规则和上下文工作的协作者。

```
# AGENT.md
## 角色
你是 Java 微服务开发助手

## 目标
分析需求、修改代码、补充测试

## 规则
- 高风险变更先出方案
- 不修改生产配置与密钥
- 公共接口变更必须补测试

## 常用命令
mvn test
mvn -Dtest=UserServiceTest test
```

一句话：/init 不是仪式感，而是把“团队怎么干活”写进 Agent 的长期记忆。

AGENT.md 主要让 Agent 记住什么?

角色与目标

你是谁，要帮团队完成什么

边界与禁区

哪些文件/操作不能碰，高风险先确认

可用工具

函数、API、MCP Server、命令如何调用

项目背景

架构、目录、术语、业务规则、技术栈

编码规范

分层规则、测试策略、提交/输出要求

安全与格式

降低幻觉，统一输出，方便协作

在 Coding Agent 场景里：

AGENT.md 负责“告诉 Agent 怎么在这个项目里做事”；

OpenSpec 负责“告诉 Agent 这次需求到底要做什么”。

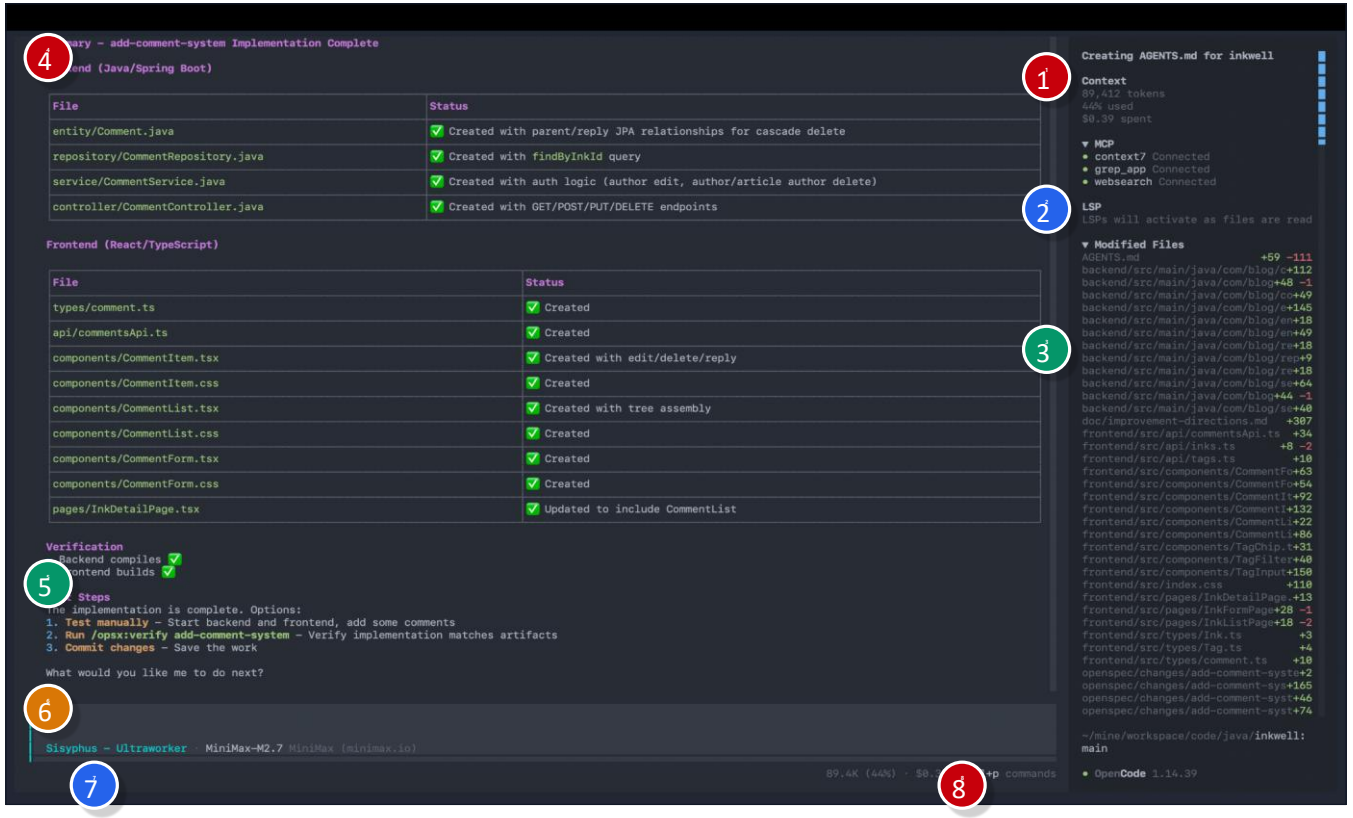
落地建议：新项目先 /init 建立 AGENT.md；每次需求先分析和计划；确认后再让 Agent 按规则改代码、跑测试、输出影响范围。

从 opencode CLI 界面理解 Agent 核心概念

用一次真实任务界面，把“上下文、工具、反馈、文件变更、人工验收”等概念落到开发过程里。

opencode CLI 任务完成界面示例

左侧是执行结果与下一步建议，右侧是上下文、工具连接和变更文件列表。



截图中的 8 个概念

这些名词可以直接对应到 Agent 的工作链路。

1. **Context**
   当前任务消耗的上下文窗口；决定 Agent 能“看见”多少代码、文件和历史信息。
2. **MCP / Tool Access**
   连接外部工具或能力，如 grep、websearch 等；Agent 通过工具执行动作。
3. **Modified Files / Diff**
   本次任务产生的文件变更与增删行数，是后续 Review 的入口。
4. **Task Result**
   主区域汇总 Agent 完成了什么，例如后端类、前端组件、接口与页面。
5. **Verification**
   编译、构建、测试等验证结果；让 Agent 的输出不只“看起来对”。
6. **Next Steps**
   Agent 给出人工下一步：手动测试、运行 verify、提交变更。
7. **Session / Model**
   底部状态展示当前会话与模型；一次任务会话会保留上下文。
8. **Project Root**
   右下角路径表示 Agent 当前进入的项目目录，也是代码库上下文边界。

关键：Agent 不是“一个更会聊天的终端”，而是围绕上下文、工具调用、执行反馈和人工验收工作的开发协作者。

模型与输入：AI Agent 到底“看见”了什么?

理解 Agent 的第一步，是理解它如何接收任务、读取信息，并在有限上下文中做判断。

名词：LLM、Prompt、System Prompt、Context、Context Window、Token

LLM

Agent 背后的“大脑”。它负责理解语言与代码、推理任务意图，并生成回答或下一步动作。

比喻：像一个能力很强的临时队友，但只能基于当前看到的信息工作。

Prompt

你交给模型的任务说明。好的 Prompt 通常会把目标、约束和输出格式一起说清楚。

比喻：像任务单；不是只说“去做”，还要说“做到什么程度、按什么方式交付”。

System Prompt

更高优先级的行为规则，用来规定角色、边界和输出方式。冲突时，System Prompt 高于普通用户指令。

比喻：像团队规范和岗位职责；任务单不能违反公司制度。

Context

模型当前能看到的资料包，可能包括代码、日志、README、AGENTS.md、测试结果和历史对话。

比喻：像桌上摊开的资料夹；资料越准，Agent 越不需要靠猜。

Context Window

模型一次能看到的“桌面大小”。窗口越大，能同时摆上的代码、文档和日志越多。

比喻：不是桌子越大越好，关键是把真正相关的材料摆上去。

Token

模型处理文本时的基本单位。代码、日志、文档都会被切成 Token，它会占用上下文容量。

比喻：像把整份资料切成小纸片；纸片太多，会挤掉真正关键的信息。

对 Coding Agent 来说，真正重要的不是只换一个更强的模型，而是把任务说清楚、把上下文给准确、把规则边界讲明白。

AI Agent 扩展机制：连接工具、沉淀能力、分工协作

MCP 解决“接什么工具”，Skill 解决“怎么稳定做事”，Subagent 解决“谁专门负责哪类任务”。

重点区分 **Skill = 可复用做事方法包 / SOP；Subagent = 被主 Agent 调度的专职小助手。前者沉淀流程，后者承担分工。**

MCP

MCP 像给 Agent 装上的“通用插座”。以前每接一个工具都要单独适配一次；有了它，Agent 可以按统一协议去连外部世界。
例子：把 GitHub、数据库、内部知识库接给 Agent，像给电脑插上扩展坞。

MCP Server

MCP Server 是真正“会干活”的那一端。它把某个系统的能力包装出来，等 Agent 来调用。
例子：一个 GitHub Server，负责查 issue、读 PR、发评论；一个数据库 Server，负责查表结构。

Tool

Tool 是 Agent 手里的“工具箱动作”。它不是知识本身，而是能真正执行的一个动作按钮。
例子：读文件、跑命令、查数据库、调用 API；像扳手、螺丝刀、万用表。

Resource

Resource 更像“参考材料架”。它提供可读信息，帮助 Agent 理解上下文，但不一定直接执行动作。
例子：README、API 文档、数据库 schema、业务规则说明；像案头摊开的资料夹。

Skill

Skill 是“做事方法包”。如果说模型会思考，那 Skill 就是在告诉它：这类活最好按什么套路做。
例子：PR Review Skill、周报生成 Skill；像新同事随身携带的一套工作 SOP。

SKILL.md

SKILL.md 是 Skill 的说明书，写清楚步骤、规则、模板和示例。它让 Agent 不只是会做，还能做得更稳定。
例子：先看 diff → 再查风险点 → 最后按模板输出 review 结论；像操作手册。

Subagent

像一个“专职角色”。主 Agent 可以把搜索、测试、代码审查、文档整理等子任务交给不同 Subagent。
例如：测试专家、代码审查员、文档整理员、代码库搜索员。

Guardrails

Guardrails 是“安全护栏”。它规定哪些动作不能直接做，哪些一碰到就必须先停下来问人。
例子：不改密钥、不碰支付逻辑、不跳过测试；像高速公路边的护栏，防止一把方向盘打飞。

Human-in-the-loop

Human-in-the-loop 表示关键节点上始终有人类参与：确认目标、Review 结果、决定是否放行。
例子：Agent 先改代码并给出 diff，人类再审查和验收；像自动驾驶也仍需要驾驶员随时接管。

一句话：MCP 让 Agent 接上外部世界；Skill 让 Agent 形成稳定方法；Subagent 让 Agent 学会分工协作；Guardrails 和人工 Review 保证可控。

一次真实请求，AI Agent 是如何工作的?

用一个开发需求，把 Prompt、Context、LLM、Skills、MCP 和 Human-in-the-loop 串成一条可讲清的链路。

用户请求： “请帮我给用户列表接口增加分页能力，兼容旧调用，补充单元测试，并说明可能影响哪些文件。”

Prompt
任务单
目标、约束、交付物
说清楚

Context
资料包
代码、AGENT.md、README
测试结果与历史约定

LLM
大脑
理解需求、判断意图
拆解关键问题

Skills
做事套路
按团队 SOP 处理
如测试/Review/变更说明

MCP
工具插座
连接文件、终端、GitHub
数据库或知识库

Human-in-the-loop
人工拍板
Review diff、确认风险
决定是否合入

ReAct 核心循环：让 LLM 一边推理，一边调用工具验证

Reason
想一想

Act
做一下

Observe
看结果

不是“一次性猜答案”，而是循环：先推理 → 调工具 → 看结果 → 再推理。
例：先找 UserController；再读 Service；再跑 mvn test；失败后继续修正。

这个流程里，每个概念各司其职

Prompt	告诉 Agent：这次要完成什么、不能破坏什么、最后要交付什么。
Context	告诉 Agent：项目当前是什么样，团队规则与历史约定是什么。
LLM	负责理解需求、做判断、在 ReAct 循环中决定下一步。
Skills	把团队经验变成固定套路，例如“先出计划、补测试、写变更说明”。
MCP	让 Agent 真正连接工具：读文件、查仓库、跑命令、访问外部系统。
Human-in-the-loop	最后由人审查结果、判断风险、决定是否接受。

一句话：Prompt 给目标，Context 给材料，LLM 负责推理，Skills 给套路，MCP 连接工具，最后由 Human-in-the-loop 把关。

SECTION

第三部分 | OpenSpec：规格驱动需求开发

从需求澄清、建立变更、实现验证，到最终归档沉淀

从知识到流程，从工具到方法

为什么不能直接让 Agent 做需求开发?

直接让 Agent “开写”很诱人，但真正的痛点通常不在“会不会写代码”，而在“需求有没有对齐、边界有没有说清、结果能不能验证”。

常见后果：写得很快，但方向跑偏；局部正确，但全局不对；代码能改出来，但不敢直接上线。

1 需求太模糊，Agent 只能靠“猜”

很多需求表面上像一句话就能说清，实际上缺少目标、约束和交付件。Agent 一旦缺信息，就会自动补全空白，而“猜得像”不等于“做得对”。

例子：“帮我优化一下用户列表接口。”——到底是加分页、加筛选、提升性能，还是要调整返回字段?

2 范围边界不清，最容易越改越多

人类开发者会下意识追问“这次改到哪儿为止”，但 Agent 如果没有明确边界，往往会把自己认为相关的内容一并改掉，造成需求蔓延。

例子：你只想“给列表接口加分页”，Agent 可能顺手改了 DTO、导出接口、前端字段、甚至接口文档。

3 隐性业务规则没有写出来，最容易犯“看似合理”的错

业务系统里真正难的，往往不是写一个分页，而是那些写在脑子里、没写在需求里的规则：租户隔离、权限判断、状态流转、历史兼容。

例子：用户列表默认只能看本部门数据；Agent 如果不知道这条隐规则，就可能把跨部门数据也查出来。

4 验收标准缺失，Agent 不知道“什么叫完成”

如果没有清晰的完成定义，Agent 很可能在“代码能跑”时就宣布结束，但团队真正关心的可能是：测试补齐了吗？兼容老调用了吗？性能有没有退化?

例子：需求说“补个能力”，但没说是否要补单测、是否要更新文档、是否要求接口耗时不明显上升。

5 高风险决策不适合让 Agent 自己拍板

涉及数据库结构、权限模型、事务边界、消息幂等、支付与生产配置时，问题已不只是“写代码”，而是“做决策”。这类决策需要人类承担责任。

例子：为了支持分页，Agent 直接建议改表结构或调整索引；技术上未必错，但是否值得做、影响多大，不应由它单独决定。

6 缺少规格说明时，协作会失真、复用也很差

今天让 Agent 改出来了，明天换一个人、换一个模型，可能又从头再猜一遍。没有需求规格，团队难以复盘，也难以让后续实现保持一致。

例子：同一句“支持分页”，A 让 Agent 返回 page/pageSize，B 让 Agent 返回 offset/limit，最后接口风格不统一。

结论：直接 Prompt ≠ 完整需求。真正需要补上的，是“范围、规则、验收标准、风险边界”——这正是后面要引出的 OpenSpec 的价值。

OpenSpec 是什么：把需求变成 Agent 可执行的规格

OpenSpec 是面向 AI Coding Assistant 的轻量级 Spec-driven Development 框架：先对齐“要构建什么”，再让 Agent 去实现。

它解决什么问题?

直接对 Agent 说“帮我实现需求”，容易出现：
- 需求只在聊天里，换会话就丢
- Agent 自行脑补边界和验收标准
- Review 只能看代码，看不到意图
- 需求、设计、任务、实现难以追溯

核心：先 Agree，再 Build。

OpenSpec 的 Skill 是什么?

它不是替代 opencode / Cursor / Claude Code 的新 Agent，
而是把一套“规格驱动开发流程”注入到已有 AI 工具中：
- Slash Commands：如 /opsx:propose、/opsx:apply、/opsx:archive
- Skills / Instructions：告诉 AI 如何生成 proposal、spec、design、tasks
- Project Context：把规格文件放进仓库，成为持久上下文

```
/opsx:propose 给用户列表接口增加分页能力
```

OpenSpec 放在哪里?

规格文件和代码一起放在仓库中，成为团队共享的“需求事实源”。

```
openspec/
├─ specs/        当前系统能力的长期规格
├─ changes/      正在进行的变更
│  └─ add-pagination/
└─ changes/archive/  已完成变更的历史记录
```

好处：需求不会随聊天窗口消失。

四个核心文档：从“为什么做”到“如何实现”

1 proposal.md
Why / What / Impact
- 为什么要做这个变更
- 要改变什么能力
- 影响范围与风险提示

2 specs//*.md**
Requirements Delta
- 用 ADDED / MODIFIED / REMOVED 描述需求变化
- 写清 Scenario / 验收行为
- Review 看意图，不只看代码

3 design.md
How / Trade-off
- 技术方案、接口与数据结构
- 兼容性、风险、备选方案
- 可跳过，但复杂需求很有用

4 tasks.md
Implementation Checklist
- 把设计拆成可执行小任务
- 每项任务可验证、可勾选
- 指导 Agent 按步骤实现

标准流程：proposal → specs → design → tasks → implement → archive

理解方式：proposal 说清“为什么与做什么”；specs 规定“系统应该表现成什么样”；design 说明“怎么做与权衡”；tasks 把它拆成 Agent 可执行清单。

用 OpenSpec 驱动一次需求开发（上）：探索 → New → Continue

示例需求：给用户列表接口增加分页能力，兼容旧调用，补充单元测试，并说明可能影响哪些文件。

1 探索 Explore：先把问题问清楚，不急着写代码

```
/opsx:explore 用户列表接口需要增加分页能力；先分析现状并列出待确认问题
```

我们在做什么：
让 AI 先读代码库、看现有接口设计、找调用链和历史行为，目的是澄清需求、识别歧义、发现风险点。此时不创建正式变更文件，也不修改代码。
生成了什么：
一份探索结果或澄清清单，例如：分页参数用 page/pageSize 还是 offset/limit？page 从 0 还是 1 开始？不传分页参数是否保持旧行为？是否需要返回 total？是否影响导出接口和前端调用？
具体例子：
它像需求评审前的“白板讨论”：先把坑挖出来，再决定如何建 spec，避免 Agent 一边写一边猜。

2 New：创建一个正式变更空间，让需求进入仓库

```
/opsx:new add-user-list-pagination
```

我们在做什么：
当需求方向基本明确后，创建一个新的变更目录，把需求从聊天窗口搬进仓库，让它成为团队共享、可追溯的工作对象。
生成了什么：
生成 openspec/changes/add-user-list-pagination/，其中通常包含 proposal.md、design.md、tasks.md，以及 specs/ 目录。proposal 讲“为什么做”；design 讲“怎么做”；tasks 讲“拆成哪些事”；specs 讲“系统行为要变成什么样”。
具体例子：
这一步像“立项建文件夹”：从这一刻起，需求不再只是几句 Prompt，而是一个有编号、有文档、有上下文的正式变更。

3 Continue：继续补全规格，把模糊需求变成清晰要求

```
/opsx:continue add-user-list-pagination；补充默认值、兼容策略、验收场景和任务拆解
```

我们在做什么：
这一阶段不是让 Agent 立刻写代码，而是继续打磨 proposal、specs、design、tasks，把边界、规则、验收标准、影响范围说完整。
生成了什么：
更新后的 proposal 会写清 Why / What / Impact；spec.md 会增加 Scenario，如“未传分页参数时保持旧行为”“传入 page=1,pageSize=20 时返回第一页数据并包含 total”；tasks.md 会拆出 DTO、Service、测试、文档等实现任务。
具体例子：
例子：把“支持分页”具体化成“page 默认 1、pageSize 默认 20、最大 100、返回 total、补正常/边界/兼容性测试”。

这一页的关键词：Explore 负责“问清楚”，New 负责“建空间”，Continue 负责“把规格补完整”。

用 OpenSpec 驱动一次需求开发（下）：Apply → Verify → Archive

继续沿用示例：add-user-list-pagination —— 即“给用户列表接口增加分页能力并保持兼容”。

4 Apply：让 Agent 按 tasks.md 去实现，而不是自由发挥

```
/opsx:apply add-user-list-pagination
```

我们在做什么：
当规格已经基本确认后，才让 Agent 开始改代码。它会以 tasks.md 为执行清单，按步骤落实 DTO、Controller、Service、测试、文档等改动。
生成了什么：
会真正修改代码并产生 diff，例如：UserController.java、UserQueryDTO.java、UserService.java、UserServiceTest.java、docs/api/user-list.md；同时 tasks.md 会从 [ ] 逐项变成 [x]。
具体例子：
它像“照着施工图施工”：Agent 干的不是脑补出来的活，而是前面被规格约束过的活。比如先给 DTO 增加 page/pageSize，再补 Service 分页逻辑，再补单测。

5 验证 Verify：确认代码、测试和规格是一致的

```
/opsx:verify add-user-list-pagination；或执行 openspec validate --strict + mvn test
```

我们在做什么：
验证阶段不是简单看编译是否通过，而是要确认：代码是否实现了 spec？测试是否覆盖了 scenario？改动是否超出了原定范围？是否还存在兼容风险？
生成了什么：
会生成一份验证结果：例如 OpenSpec 规格校验通过、UserServiceTest 通过、未传分页参数时旧行为保持兼容、pageSize 超限时有保护逻辑；同时也会暴露问题，如“新增 total 可能影响某些旧调用方反序列化”。
具体例子：
它像“验收测试 + 对账”：不是只看“能不能跑”，而是看“是不是按说好的方式在跑”。

6 归档 Archive：把临时变更沉淀为系统事实

```
/opsx:archive add-user-list-pagination
```

我们在做什么：
需求通过 Review 并完成交付后，执行归档。OpenSpec 会把完成的 change 移入 archive，并把临时规格合并到长期 specs 中，形成新的“当前系统事实”。
生成了什么：
changes/add-user-list-pagination/ 会变成 changes/archive/2026-05-xx-add-user-list-pagination/；而变更后的行为描述会沉淀到 openspec/specs/user-list/spec.md 中。
具体例子：
这一步像“关闭工单并更新系统说明书”：以后再有人问‘用户列表分页为什么这么设计’，不用翻聊天记录，直接看 archive 和 specs 就能追溯。

一句话总结：OpenSpec 让需求开发从“一句 Prompt 直接开写”，变成“先探索 → 建变更 → 补规格 → 再实现 → 再验证 → 最后沉淀”的闭环。

SECTION

第四部分 | 赋能总结：价值、误区与拥抱方式

让 AI 多干活，但别让 AI 独自拍板；让 AI 提效率，但别让 AI 替你负责

从知识到流程，从工具到方法

如何看待 AI 辅助研发：能力放大器，而不是自动驾驶

核心观点：AI 的价值不是替我们“少思考”，而是把重复劳动交给机器，把问题定义、方案判断、质量把关留给人。

价值 AI 能为研发带来什么？

- 降低重复劳动：生成样板代码、补单测、写 PR 描述、整理日志分析。
- 提升理解效率：快速梳理陌生模块职责、核心入口、调用链和风险点。
- 前置质量检查：帮助发现边界条件、异常分支、兼容性影响和测试遗漏。
- 加速需求闭环：从需求澄清、规格生成、任务拆解到实现、验证、Review 辅助。
- **本质上：把“机械劳动”压缩，让开发者把精力放回问题定义与工程判断。**

误区 对 AI 的常见误区

- “AI 会写代码，所以能替代开发者”——错。真正难的是需求、边界、责任和长期维护。
- “AI 说得很自信，所以它一定是对的”——错。表达流畅，不代表结论可靠。
- “上下文越多越好”——不一定。关键不是喂得多，而是喂得准、喂得相关。
- “有了 Agent 就能跳过 Review”——恰恰相反。Agent 越能执行，越需要边界、测试和人工验收。
- **一句话：AI 会替代一部分机械动作，但不会替代工程责任。**

姿势 我们应该如何拥抱 AI？

- 把 AI 当“研发搭子”，不是外包团队：你仍然是负责人，方向盘还在你手里。
- 先让 AI 分析和计划，再让它执行：先问理解、影响范围、风险点、测试方案，再改代码。
- 把需求说清楚，把上下文给准确：目标、背景、范围、约束、验收标准，缺一不可。
- 用工程机制驯服 AI：AGENTS.md、OpenSpec、tasks.md、单测、Lint、Review、Guardrails。
- **真正会用 AI 的开发者，是更会定义问题、组织上下文、验证结果的人。**

AI 并不是完全可信正确的

互联网笑话：AI 有时候像一个刚入职三天、但语气像 CTO 的实习生。

另一个经典说法：AI 最大的问题不是它会错，而是它错的时候也很像对的。

因此要记住：

- AI 输出的是建议，不是结论
- AI 生成的是候选方案，不是最终方案
- AI 写出的代码，需要测试、Review 和人类判断

结论：让 AI 多干活，但别让 AI 独自拍板；让 AI 提效率，但别让 AI 替你负责。

软件开发视角：AI 擅长什么，不适合什么？

这一页补充一个使用边界：把 AI 用在它擅长的地方，用工程机制兜住它不适合独立承担的地方。

AI 擅长做什么？

适合“有上下文、有规则、可验证”的开发任务

- 读懂陌生代码：梳理模块职责、调用链、关键类和潜在风险。
- 生成低风险代码：样板代码、DTO、参数校验、工具函数、适配层。
- 补测试与边界用例：空值、异常、边界、兼容性场景。
- 解释报错和日志：根据堆栈、日志、最近改动定位可能原因。
- 做变更说明：整理影响范围、修改文件、测试结果和 Review 要点。
- 执行“有 SOP 的任务”：例如 PR Review、接口文档初稿、重构建议。

关键词：明确目标 + 准确上下文 + 可运行测试 = AI 更容易稳定产出。

AI 不适合做什么？

不适合“缺上下文、强决策、高风险、难验证”的事情

- 独立拍板架构方案：微服务边界、技术选型、长期演进路线。
- 直接修改高风险逻辑：权限、支付、账务、数据删除、生产配置。
- 猜业务隐性规则：租户隔离、历史兼容、审批流程、特殊客户逻辑。
- 无测试地大规模重构：跨模块改动、公共接口行为变化、数据库迁移。
- 代替人工验收：它可以自测，但不能替团队承担上线责任。
- 处理模糊需求直接开写：需求没对齐时，越快越容易跑偏。

关键词：高风险决策必须 Human-in-the-loop；方向不清先回到需求和方案。

使用原则：让 AI 做“资料整理、代码生成、测试补充、自检说明”；让人负责“需求判断、方案取舍、风险验收、最终上线责任”。

SECTION

第五部分 | AI Agent 协作开发范式

从“计划 - 实施 - 验证 - 学习”的四段式，看人与 AI 如何高效协作

人负责决策与把关，AI 负责方案、实施与自测

AI 结对编程：计划 - 实施 - 验证 - 学习

这套协作范式强调：人始终承担决策与验收职责，AI 负责需求复述、方案生成、增量实施与自测。

核心精神：先对齐、再决策、再增量实施、再验证；方向错了回到方案，局部问题走修复小循环。

阶段 1 | 需求理解与任务拆分（对齐）

人：提供业务背景、目标、约束条件和优先级，可附带示例或接口定义。
AI：进行需求复述、识别歧义，提出分解后的子任务草案，并列出每个子任务的边界、前后依赖、验收标准草稿。
人：确认、调整拆分结构，标注哪些子任务优先实施，明确终止条件（Done criteria）。
此处若发现原始需求本身不清，则回到本阶段起点补充信息。

阶段 2 | 方案设计与风险评估（决策核心）

AI：针对当前子任务，生成**可选解决方案**（含技术选型、核心流程伪代码 / 架构草图），并附带：
每种方案的优缺点对比
关键风险点（安全、性能、兼容性、数据一致性等）
待决策事项（需要人拍板的技术取舍、资源权衡）
人：审查方案，关注风险可接受性，对关键决策做出选择，必要时要求 AI 修补漏洞或补充备选方案。
此处设置硬 Gate：人明确说“按此方案执行”，AI 才能进入下一步。若方案不可行，则循环本阶段。

阶段 3：验证策略制定（测试先于实施）

AI：基于已确认的方案，产出**可执行的验证策略**，包括：
自动化测试用例生成（单元 / 集成 / 端到端）
人工验证清单（边界情况、UX 行为、安全扫描）
验证通过的标准（覆盖率阈值、性能基准等）
人：复核验证策略是否覆盖风险点与业务需求，批准后进入实施。

阶段 4：增量实施与自检（AI 主导，人可随时介入）

AI：按以下节奏实施：
生成代码 & 必要配置；
立即运行已生成的验证用例（或快速自检脚本）；
将运行结果、覆盖率、自查出的问题一并输出
若 AI 自检发现问题：AI 自动尝试修复（设定重试上限），超出能力则暂停并请求人介入。
人：在 AI 增量完成一个逻辑单元后，进行**快速 review**（可在 IDE 中查看 diff），重点检查逻辑正确性、安全性与设计一致性。发现问题即时给出定向反馈，进入“局部修正循环”。

阶段 5：人工验证与决策（门禁）

人：运行完整验证套件，结合人工验证清单做验收测试。
通过：子任务完成，打标签，进入下一个子任务。
发现问题：描述错误现象、上下文，反馈给 AI。
AI：基于反馈进行**上下文保留式修正**（不丢失已有方案与已澄清需求），修正后再次经过自检并提交给人验证。此局部循环最多重复 N 次，避免无底循环。若反复不过，上升为方案变更，回退到阶段 2 重新设计。

阶段 6：闭环与沉淀（持续改进）

AI：总结本次子任务中实际遇到的风险、做出的决策、代码中需要注意的设计约束，形成一条**开发笔记**，供团队和后续 AI 调用时参考。
人：确认归档，必要时补充隐性知识。

核心要点：① 人负责决策与验证，AI 负责方案、实施与自测； ② 局部修正循环高效且不丢上下文； ③ 只有方向错误时，才触发大循环回到方案阶段。

AI Agent 协作开发范式 | 简洁版流程图

这页用简笔画展示一条最适合日常开发落地的协作主线：对齐 → 方案 → 执行 → 验证，并保留两个反馈回路。

= 人：决策 / 把关

= AI：方案 / 实施 / 自测

= 通过进入下一步；不通过则走反馈回路

1. 人给背景与目标

提供业务背景、目标、约束
和优先级

2. AI 复述并拆任务

识别歧义，拆成子任务
列边界与验收草稿

3. AI 生成方案、风险、待决策

给可选方案、风险、影响范围
和推荐理由

4. 人确认方案

通过则批准；不通过
反馈修改

方案不通过 → 反馈修改

5. AI 增量实施 + 自测

按批准方案改代码
并运行自测

6. 人验证

看结果是否满足预期
是否可接受

7. 完成 / 沉淀

整理结论，沉淀经验到
规则或知识库

验证后有问题?

- 局部问题：回到“AI 增量实施 + 自测”
- 方向错了：回到“AI 生成方案”

这套范式最重要的不是“让 AI 一次做完”，而是让人与 AI 形成稳定节奏：先对齐、再决策、再增量实施、再验证，必要时小步回退。

END / Q&A

谢谢聆听

欢迎交流：AI 不是替你负责的人，而是放大你能力的工具。

“让 AI 多干活，但别让 AI 独自拍板。”

- 问题 Q
- 交流 A
- 讨论 &